

ISCM

Information System Compliance Monitor

Technical Architecture Document

Status	Checks	SubControls	Tests
Production Ready	22/22 (100%)	119/119 (100%)	88 Passing

Technology Stack: C# .NET 8 · xUnit · Moq · FluentAssertions
Architecture Pattern: Collector-Only with Typed Evaluation Pipeline
Platform: Windows (Registry, PowerShell, WMI, auditpol, secedit)
Document Date: September 2026
Version: 1.0 — Post Phase 11 Final Migration

This document provides a complete reference for the ISCM project architecture, implementation details, design patterns, migration journey, testing strategy, and operational guidance.

Table of Contents

- 1. Executive Summary
- 2. Project Overview
 - 2.1 Purpose
 - 2.2 Key Achievements
- 3. Architecture Layers
 - 3.1 Domain Layer (ISCM.Domain)
 - 3.2 Application Layer (ISCM.Application)
 - 3.3 Infrastructure Layer (ISCM.Infrastructure)
 - 3.4 Tests Layer (ISCM.Tests)
- 4. Data Flow Diagram
- 5. Key Design Patterns
 - 5.1 Collector-Only Pattern
 - 5.2 Typed Evaluation Pipeline
 - 5.3 Catalog-Driven Validation
- 6. Migration Journey
 - 6.1 Phase 10: Scanner Execution
 - 6.2 Phase 11: Final Migration
- 7. Evidence Collection Methods
- 8. Catalog Structure
- 9. Testing Strategy
- 10. Technical Debt Resolution
- 11. Gate Criteria
- 12. Branch History
- 13. Future Enhancements
- 14. Appendix
 - 14.1 File Structure
 - 14.2 Key Metrics
 - 14.3 Dependencies
 - 14.4 Build Commands
 - 14.5 Migration Checklist

1. Executive Summary

ISCM (Information System Compliance Monitor) is an automated Windows security hardening compliance scanner written in C# on .NET 8. It evaluates target systems against a defined set of security baselines and produces structured compliance findings with a full evidence trail.

The system implements a strict **collector-only pattern** combined with a **typed evidence evaluation pipeline**. Individual checks are responsible solely for acquiring raw evidence; all evaluation logic lives in a centralized, catalog-driven pipeline that uses strongly-typed value objects and operator-aware comparison.

Metric	Value	Notes
Project Name	ISCM (Information System Compliance Monitor)	Windows security hardening scanner
Technology Stack	C# .NET 8, xUnit, Moq, FluentAssertions	Windows-only runtime
Checks Migrated	22 / 22 (100%)	All collector-only
SubControls Defined	119 / 119 (100%)	Catalog complete
Tests Passing	88 / 88 (100%)	Unit + Integration
Architecture Pattern	Collector-only + Typed Evaluation Pipeline	Zero legacy code remaining
Evidence Sources	Registry, PowerShell, WMI, auditpol, secedit, net	Multi-source capable

As of the completion of Phase 11, the project has achieved full migration of all checks, complete catalog definition, removal of all legacy evaluation paths, and a clean, maintainable architecture ready for production use and future extension.

2. Project Overview

2.1 Purpose

ISCM scans Windows systems against a comprehensive security baseline consisting of 17 baseline controls derived from hardening guidance plus 3 extended checks. It produces compliance findings that include the current value, expected value, evaluation status, and a complete evidence trail showing how each value was obtained.

Core capabilities:

- Scans against 17 baseline controls + 3 extended checks (119 SubControls total)
- Produces compliance findings with full evidence trail (raw output + typed value)
- Supports multiple evidence sources: Registry, PowerShell, WMI, auditpol, secedit, net commands
- Evaluates evidence using a typed pipeline driven by catalog metadata
- Enforces single-responsibility collector-only design for all checks

2.2 Key Achievements

Phase	Achievement	Result
Phase 10	Migrated 18 checks to typed pipeline	81.8% complete
Phase 11	Migrated final 4 checks (ADM-001, DEF-001, USB-001, ALG-001) + legacy removal	100% complete
Architecture	100% collector-only pattern enforcement	Zero legacy code remaining
Quality	88 tests passing with 100% success rate	Build: 0 errors

3. Architecture Layers

ISCM follows a clean layered architecture with clear separation of concerns:

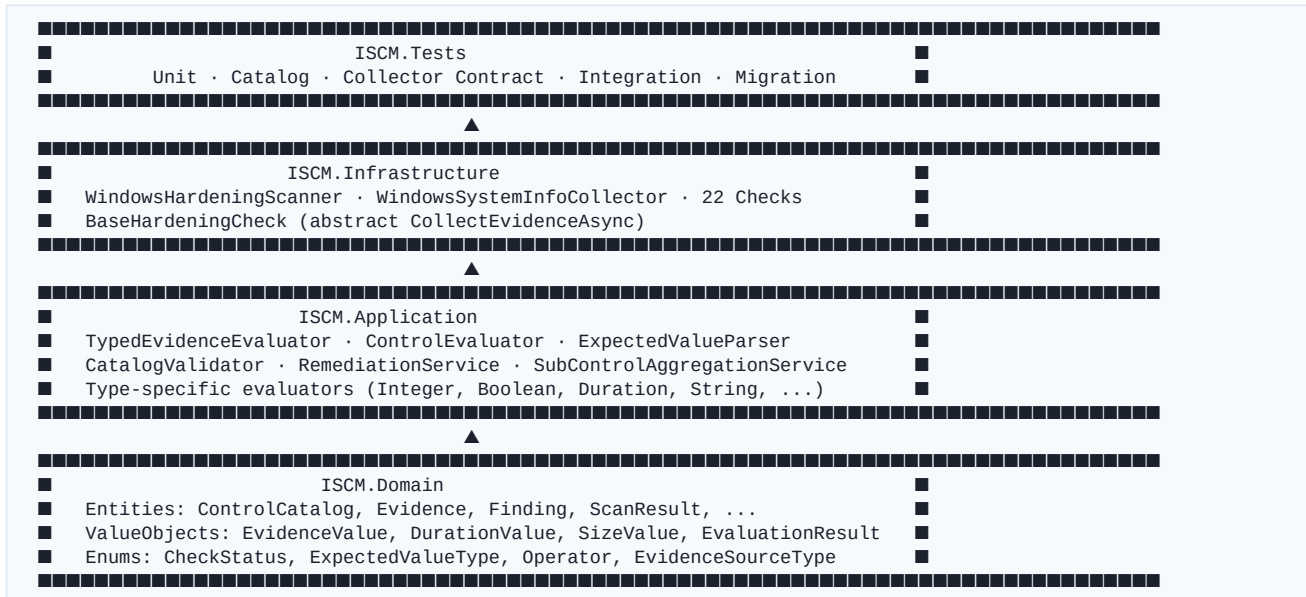


Figure 1: High-level layered architecture

3.1 Domain Layer (ISCM.Domain)

The Domain layer contains pure business concepts with no infrastructure dependencies. It defines the core entities, value objects, and enumerations that model the compliance domain.

Entities

Entity	Description
ControlCatalog	Root catalog containing 17 baseline + 3 extended controls and 119 SubControls
ControlDefinition	Parent control metadata: ControlId, Title, Category, Severity, TechnicalCheckIds
SubControlDefinition	SubControl metadata: SubControlId, SettingName, ExpectedValue, ExpectedValueType, Operator, EvidenceSources
Evidence	Raw output + TypedValue + metadata (ScanId, PathId, SourceType, AcquisitionCommand)
Finding	Final compliance result: CheckId, Status, CurrentValue, ExpectedValue, Description
SubControlResult	Intermediate result with EvidenceItems and evaluation status
ScanResult	Complete scan output with findings and overall compliance score

Value Objects

Value Object	Purpose
EvidenceValue	Typed wrapper supporting Integer, Boolean, Duration, String, Enum, Collection, RegistryValue, PolicyValue
DurationValue	Duration with unit (Days, Hours, Minutes, Seconds)
SizeValue	Size with unit (KB, MB, GB)
EvaluationResult	Pass / Fail / Error with reason and details

Enumerations

Enum	Values
CheckStatus	Pass, Fail, Unknown, Error, NotScanned
ExpectedValueType	Integer, Boolean, Duration, String, Enum, Collection, RegistryValue, PolicyValue

Enum	Values
Operator	Equals, NotEquals, GreaterThan, GreaterOrEqual, LessThan, LessOrEqual, Contains, SetMembership
CheckCategory	Account, System, Network, Audit
CheckSeverity	Critical, High, Medium, Low
EvidenceSourceType	Registry, PowerShell, Wmi, Auditpol, Secedit, NetAccounts, ProcessOutput, Other, Unknown

3.2 Application Layer (ISCM.Application)

The Application layer contains the core evaluation logic, services, and contracts. It orchestrates typed evaluation, catalog validation, and aggregation of results.

Evaluators

Component	Responsibility
TypedEvidenceEvaluator	Main dispatcher that routes to type-specific evaluators based on ExpectedValueType
IntegerEvaluator	Compares integer values using comparison operators
LongEvaluator	Compares long values (used for Size comparisons)
BooleanEvaluator	Compares boolean values (Equals / NotEquals)
StringEvaluator	Compares strings (Equals, NotEquals, Contains)
DurationEvaluator	Compares TimeSpan values with full operator support
SizeEvaluator	Compares size values with unit awareness
EnumEvaluator	Compares enumeration values
CollectionEvaluator	Compares collections (Contains, SetMembership)
RegistryValueEvaluator	Specialized comparison for registry values
PolicyValueEvaluator	Specialized comparison for policy values

Services

Service	Responsibility
ControlEvaluator	Aggregates SubControl results into Findings with agreement analysis
ExpectedValueParser	Parses expected strings into typed objects (e.g., "14 characters" → 14, "60 days" → TimeSpan)
CatalogValidator	Validates catalog integrity: duplicates, orphans, typed contracts
EvidenceAcquisitionService	Abstract service for evidence collection orchestration
RemediationService	Applies fixes to non-compliant systems
SubControlAggregationService	Applies agreement / disagreement analysis across multiple verification paths

Key Interfaces

Interface	Contract
IEvidenceCollector	CollectEvidenceAsync() → List<Evidence> (collector-only checks)
IControlEvaluator	Contract for evaluation logic
ITypedEvidenceEvaluator	Contract for typed evaluation
ICatalogValidator	Contract for catalog validation
IScanService	Contract for scanner orchestration

3.3 Infrastructure Layer (ISCM.Infrastructure)

The Infrastructure layer contains Windows-specific scanning, system information collection, and all 22 hardening checks. Every check inherits from BaseHardeningCheck and implements only CollectEvidenceAsync.

Core Scanning Components

Component	Role
WindowsHardeningScanner	Main orchestrator: calls CollectEvidenceAsync, evaluates with catalog metadata, produces Findings
WindowsSystemInfoCollector	Collects system metadata (hostname, IP, MAC, OS version)
BaseHardeningCheck	Abstract base enforcing collector-only pattern (abstract CollectEvidenceAsync, no EvaluateAsync)

Complete Check Inventory (22 Checks)

#	Check Class	ID	SubControls
1	PasswordLengthCheck	PWD-001	6
2	AccountLockoutCheck	LCK-001	3
3	GuestAccountCheck	GUEST-001	2
4	AdvancedAuditCheck	AUD-001	11
5	ProcessCreationAuditingCheck	PRC-001	3
6	PowerShellLoggingCheck	PSH-001	3
7	EventLogSizeCheck	EVL-001	13
8	AutoRunDisabledCheck	ARD-001	3
9	UserAccountControlCheck	UAC-001	3
10	LmCompatibilityCheck	LM-001	2
11	CredentialGuardCheck	CRG-001	6
12	FirewallDomainProfileCheck	FW-001	16
13	LlmnrNetbiosCheck	LLN-001	7
14	SmbV1ProtocolCheck	SMB-001	5
15	WindowsUpdateCheck	WUP-001	4
16	RdpNlaCheck	RDP-001	7
17	DisableCmdCheck	CMD-001	3
18	UserRightsCheck	URA-001	9
19	AdminAccountCountCheck	ADM-001	1
20	WindowsDefenderCheck	DEF-001	3
21	UsbStorageCheck	USB-001	1
22	AutoLogonCheck	ALG-001	1

3.4 Tests Layer (ISCM.Tests)

The test suite comprises 88 tests across seven categories, achieving a 100% pass rate. The framework stack is xUnit (runner), Moq (mocking), and FluentAssertions (readable assertions).

Category	Count	Focus
Unit tests	48	Typed evaluators, parsers, comparison engines
Catalog tests	6	Integrity validation, typed contracts
Collector contract tests	5	Verify collector-only pattern enforcement
Scanner integration tests	2	Real system orchestration
Pilot migration tests	3	PasswordLengthCheck validation
Batch migration tests	16	Account, Audit, Registry, Network checks
New integration tests	8	AdminAccountCount, WindowsDefender, UsbStorage, AutoLogon
Total	88	100% pass rate

4. Data Flow Diagram

The end-to-end data flow from a check collecting evidence to a final evaluation result is strictly unidirectional and catalog-driven:

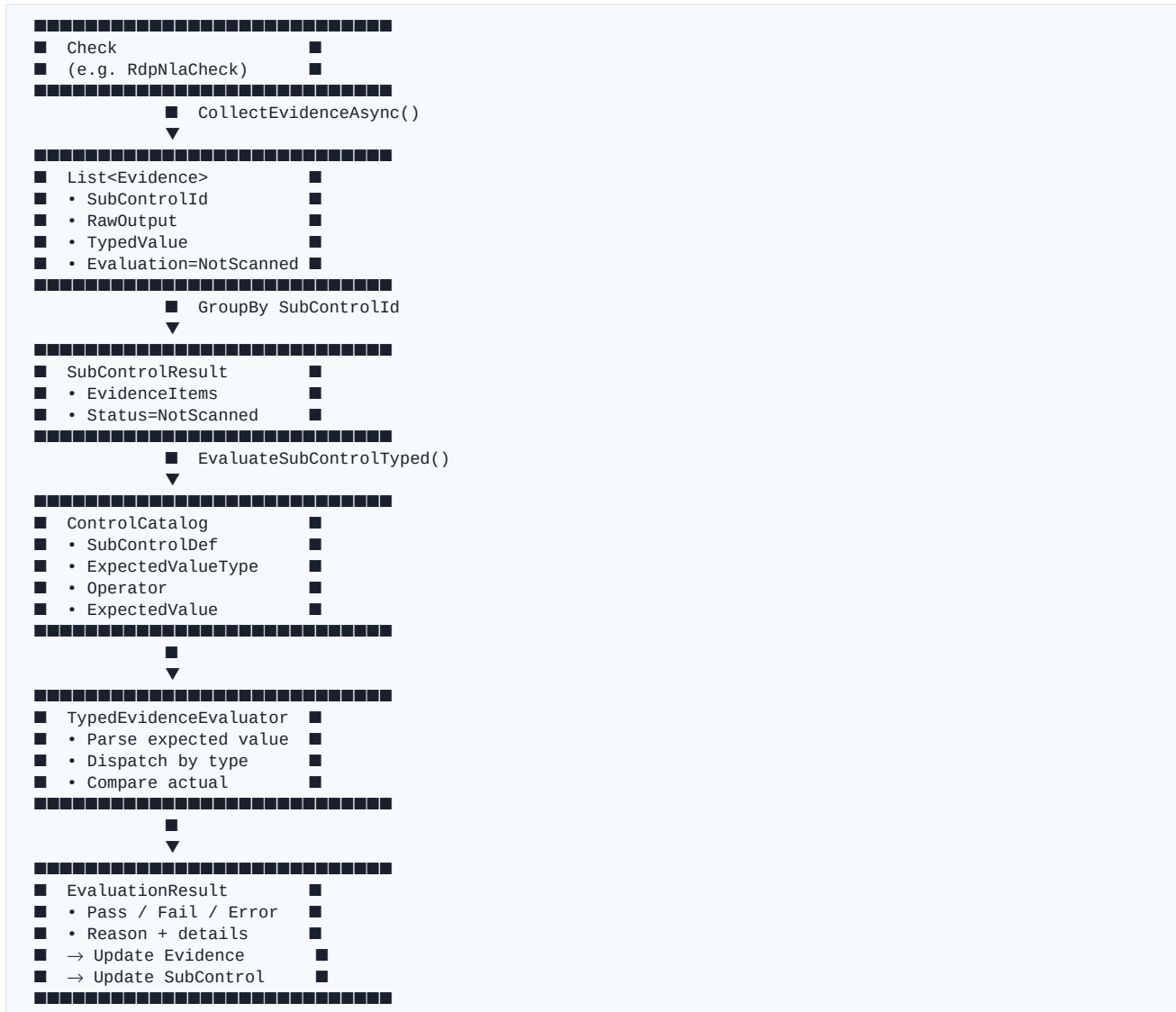


Figure 2: Evidence collection and typed evaluation data flow

Key invariants of this flow:

- Checks never perform evaluation — they only collect and type evidence.
- Evaluation is always driven by catalog metadata (ExpectedValueType + Operator).
- TypedValue is populated at collection time for compile-time and runtime type safety.
- Evidence starts with Evaluation = NotScanned; the scanner updates it after evaluation.

5. Key Design Patterns

5.1 Collector-Only Pattern (Single Responsibility Principle)

The collector-only pattern is the foundational design decision of ISCM. Checks are responsible only for acquiring data; evaluation is performed exclusively by the scanner using catalog metadata.

Before (Legacy)

```
public class MyCheck : IHardeningCheck
{
    public Task<Finding> EvaluateAsync()
    {
        var value = Registry.GetValue(...);
        var status = value == "Expected" ? CheckStatus.Pass : CheckStatus.Fail;
        return new Finding(..., status, value, "Expected", ...);
    }
}
```

After (Collector-Only)

```
public class MyCheck : BaseHardeningCheck
{
    public override Task<List<Evidence>> CollectEvidenceAsync()
    {
        var value = Registry.GetValue(...);
        return new List<Evidence>
        {
            new Evidence
            {
                SubControlId = "MY-001.1",
                RawOutput = value.ToString(),
                TypedValue = EvidenceValue.FromBoolean(value == 1),
                Evaluation = CheckStatus.NotScanned // Scanner evaluates
            }
        };
    }
}
```

Benefits

- Checks only collect data (single responsibility)
- Scanner evaluates with catalog metadata (separation of concerns)
- Type safety enforced at both compile time and runtime
- Easier to unit-test collectors and evaluators independently
- Consistent evaluation logic across all checks

5.2 Typed Evaluation Pipeline

The typed evaluation pipeline converts string expected values from the catalog into strongly-typed objects, then dispatches comparison to the appropriate type-specific evaluator.

Pipeline Steps

1. Check collects Evidence with TypedValue already populated
2. Scanner groups Evidence by SubControlId
3. Scanner calls EvaluateSubControlTyped with catalog metadata (ExpectedValueType, Operator, ExpectedValue)
4. TypedEvidenceEvaluator parses the expected string into a typed object
5. Routes to the type-specific evaluator (IntegerEvaluator, BooleanEvaluator, etc.)
6. Returns EvaluationResult (Pass / Fail / Error + reason)
7. Updates Evidence.Evaluation and SubControlResult.Status

Supported Types and Operators

Type	~Count	Example	Operator Support
Boolean	~35	EnableLUA, DisableCMD	Equals, NotEquals
Integer	~25	LmCompatibilityLevel, AUOptions	Equals, NotEquals, Gt, Gte, Lt, Lte
String	~8	WUSever, Guest account name	Equals, NotEquals, Contains
Collection	~20	Audit policies, User Rights	Contains, SetMembership
Duration	~8	Password age, MaxIdleTime	Equals, NotEquals, Gt, Gte, Lt, Lte
Enum	~6	ConsentPromptBehaviorAdmin	Equals, NotEquals, SetMembership

5.3 Catalog-Driven Validation

All SubControls are defined in ControlCatalog with complete metadata. The scanner never hard-codes expected values or operators; it always consults the catalog. CatalogValidator ensures integrity by detecting duplicates, orphans, and missing typed contracts.

- All SubControls defined in ControlCatalog with full metadata
- ExpectedValueType and Operator declared declaratively in the catalog
- Scanner uses catalog metadata exclusively for evaluation decisions
- CatalogValidator guarantees: no duplicates, no orphans, all required typed contracts present

6. Migration Journey

6.1 Phase 10: Scanner Execution (18 checks migrated)

Phase 10 established the typed pipeline, migrated the majority of checks, and fixed six critical bugs discovered during migration.

Sub-Phase	Focus
10.1	Test project setup (48 tests)
10.2	Typed catalog contract (6 tests)
10.3	Collector contract (5 tests)
10.4	Typed pipeline + 3 bug fixes (2 tests)
10.5	Pilot migration — PasswordLengthCheck (6 SubControls)
10.6	Account checks batch (5 SubControls)
10.7	Audit checks batch (23 SubControls)
10.8	System / Network checks batch (41 SubControls in 3 batches)
10.9	CMD + UserRights checks (12 SubControls)

Bugs Fixed in Phase 10

ID	Description
BUG-01	Duration pipeline bridge (DurationValue → TimeSpan conversion in DispatchDuration)
BUG-02	Duration evaluation requires TimeSpan values (not DurationValue)
BUG-03	Disagreement precedence in result combination (Error > Unknown > Fail > Pass)
BUG-04	Boolean parsing for descriptive values ("Enabled" → true)
BUG-05	Keyword-specific parsing cross-contamination (isolate parsing logic per SubControl)
BUG-06	Type mismatch between check output and catalog ExpectedValueType

6.2 Phase 11: Final Migration (4 checks + legacy removal)

Phase 11 completed the migration of the remaining four checks, performed catalog restructuring, removed all legacy code paths, and added integration tests.

Sub-Phase	Focus
11.0	Catalog restructuring (ADM-001.1/2/3 → SEC-001.6/7/8, populated EXT-01/02/03)
11.1	AdminAccountCountCheck migration (PowerShell + net localgroup + WMI)
11.2	WindowsDefenderCheck migration (Registry + PowerShell, 3 SubControls)
11.3	UsbStorageCheck migration (Registry USBSTOR\Start)
11.4	AutoLogonCheck migration (Registry AutoAdminLogon + DefaultPassword)
11.5	Legacy removal (deleted IMultiPathCheck, IMultiPathCheckValidator; simplified interfaces)
11.6	Integration tests for 4 new checks (8 tests added)
11.7	Architecture documentation

Files Deleted

- ISCM.Application/Interfaces/IMultiPathCheck.cs
- ISCM.Application/Interfaces/IMultiPathCheckValidator.cs
- ISCM.Application/Validators/MultiPathCheckValidator.cs

Files Simplified

- IHardeningCheck — reduced to metadata-only interface (CheckId, Name, Category, Severity)

- BaseHardeningCheck — abstract CollectEvidenceAsync only; no EvaluateAsync
- WindowsHardeningScanner — removed RunLegacyPath and IMultiPathCheckValidator dependency

7. Evidence Collection Methods

Checks acquire evidence from multiple Windows-native sources. Multi-source verification is supported: a single SubControl may produce multiple Evidence items that the scanner later aggregates.

Source	Technique	Examples
Registry	Registry.LocalMachine.OpenSubKey() with PowerShell Get-ItemProperty fallback	USBSTOR\Start, AutoAdminLogon, DisableRealtimeMonitoring
PowerShell	Process.Start("powershell.exe", "-Command ...")	Get-MpComputerStatus, Get-LocalGroupMember, Get-WindowsOptionalFeature
WMI	ManagementObjectSearcher with WQL queries	Win32_GroupUser for group membership
Command-Line	net accounts, net localgroup, auditpol, secedit	Password policy, group membership, audit policy, security policy
Multi-Source	Multiple Evidence items per SubControlId; scanner groups and evaluates	Agreement analysis across independent verification paths

8. Catalog Structure

8.1 Baseline Controls (17 from hardening guidance)

#	Control	SubControls
01	Password Policy	6
02	Account Lockout Policy	3
03	Disable Guest Account	2
04	Advanced Audit Policy	11
05	Process Creation Auditing	3
06	PowerShell Script Block Logging	3
07	User Rights Assignment	9
08	Security Options	13 (incl. ADM-001.1)
09	Disable CMD & Script Execution	3
10	Windows Defender Firewall	16
11	Disable LLMNR & NetBIOS	7
12	Credential Guard & LSA Protection	6
13	Disable SMBv1	5
14	Windows Update / Patch Management	4
15	Disable Autorun / Autoplay	3
16	Secure RDP	7
17	Event Log Size & Retention	13

8.2 Extended Controls (3 additional)

ID	Control	Check	SubControls
EXT-01	Windows Defender	DEF-001	3
EXT-02	USB Storage Policy	USB-001	1
EXT-03	AutoLogon Disabled	ALG-001	1

8.3 SubControl Metadata Fields

Each SubControlDefinition carries the following metadata that drives evaluation:

- SubControlId (e.g., "PWD-001.4")
- SettingName (human-readable name)
- ExpectedValue (e.g., "14 characters")
- ExpectedValueType (Integer, Boolean, Duration, ...)
- Operator (Equals, GreaterOrEqual, ...)
- Description (purpose of the control)
- Category (Account, System, Network, Audit)
- Severity (Critical, High, Medium, Low)
- IsRequired (boolean)
- ParentControlId (e.g., "01")
- EvidenceSources (list: "Registry", "PowerShell", ...)

9. Testing Strategy

9.1 Test Categories

The 88 tests are organized into seven complementary categories that together provide confidence in both individual components and end-to-end orchestration.

Unit Tests (48)

- Typed evaluators (IntegerEvaluator, BooleanEvaluator, DurationEvaluator, ...)
- ExpectedValueParser
- Comparison engines
- Catalog validation logic

Catalog Tests (6)

- No duplicate SubControlIds
- No orphan SubControls
- All required SubControls have typed contracts
- Valid ValueType / Operator combinations

Collector Contract Tests (5)

- BaseHardeningCheck must implement IEvidenceCollector
- CollectEvidenceAsync must return Evidence with Evaluation = NotScanned
- TypedValue must be populated

Scanner Integration Tests (2)

- Scanner calls CollectEvidenceAsync
- Scanner evaluates with catalog metadata
- Agreement analysis applied

Pilot Migration Tests (3)

- Real-system tests for PasswordLengthCheck
- Typed evaluation verification
- Evidence structure validation

Batch Migration Tests (16)

- Account checks (4), Audit checks (5)
- Registry checks batch 1 (4), Network checks batch 2 (4)
- Secure RDP (2), CMD and UserRights (3)

New Integration Tests (8)

- AdminAccountCountCheck (2), WindowsDefenderCheck (2)
- UsbStorageCheck (2), AutoLogonCheck (2)

9.2 Test Framework & Coverage

Aspect	Detail
Runner	xUnit 2.5.7 (Fact / Theory attributes)
Mocking	Moq 4.20.70
Assertions	FluentAssertions 6.12.0
Total tests	88
Pass rate	100%
Check coverage	All 22 checks have integration test coverage
Evaluator coverage	All type-specific evaluators have unit tests

Aspect	Detail
Catalog coverage	Full integrity validation covered

10. Technical Debt Resolution

10.1 Phase 10 Bugs Fixed

Six bugs were identified and resolved during the Phase 10 migration. See Section 6.1 for the detailed list (BUG-01 through BUG-06).

10.2 Phase 11 Legacy Removal

- Deleted IMultiPathCheck interface (no longer used)
- Deleted IMultiPathCheckValidator interface and implementation
- Simplified IHardeningCheck to metadata-only (removed EvaluateAsync / EvaluateSubControlsAsync)
- Simplified BaseHardeningCheck to abstract CollectEvidenceAsync only
- Simplified WindowsHardeningScanner to collector-only path (removed RunLegacyPath)

10.3 Remaining Compiler Warnings

Warning	Count	Notes
CS8618	10	Non-nullable properties in Domain entities (low priority)
CA1416	25	Platform-specific code in RemediationService (Windows-only, acceptable)
CS8625	1	Nullable reference type in WindowsHardeningScanner (low priority)
Total	36	0 errors

11. Gate Criteria

11.1 Gate 10 Criteria (Phase 10 Completion)

Criterion	Status
18/22 checks migrated (81.8%)	✓ Achieved
87/114 SubControls defined (76.3%)	✓ Achieved
80 tests passing (100% success rate)	✓ Achieved
Typed pipeline working for all migrated checks	✓ Achieved
Catalog validation: no Critical issues	✓ Achieved
Build successful (0 errors)	✓ Achieved

11.2 Gate 11 Criteria (Phase 11 Completion)

Criterion	Status
22/22 checks migrated (100%)	✓ Achieved
119/119 SubControls defined (100%)	✓ Achieved
0 legacy code remaining	✓ Achieved
88 tests passing (100% success rate)	✓ Achieved
Catalog validation: no Critical issues	✓ Achieved
Build successful (0 errors)	✓ Achieved
Architecture documentation created	✓ Achieved

12. Branch History

12.1 epic/phase10-scanner-execution

- Commits: 10.1 through 10.9
- Migrated 18 checks to typed pipeline
- Fixed 6 bugs
- 80 tests passing at end of phase

12.2 epic/phase11-final-migration

- Commits: 11.0 through 11.7
- Migrated final 4 checks (ADM-001, DEF-001, USB-001, ALG-001)
- Removed all legacy code
- Added 8 integration tests
- Created ARCHITECTURE.md
- 88 tests passing

13. Future Enhancements

13.1 Potential Improvements

- **Three-Path Verification** — Implement independent verification paths with agreement analysis (optional feature)
- **Remediation Engine** — Automated fix application for non-compliant systems
- **Reporting & Dashboard** — Professional output formats (PDF, HTML, JSON)
- **Performance Optimization** — Parallel scanning, caching, batch operations
- **CI/CD Pipeline** — Automated testing and deployment
- **Code Coverage** — Add coverage reporting (Coverlet, ReportGenerator)

13.2 Technical Debt Backlog

- Fix 10 CS8618 warnings (non-nullable properties)
- Fix 25 CA1416 warnings (platform-specific code)
- Fix 1 CS8625 warning (nullable reference type)
- Add XML documentation comments to all public APIs
- Add performance benchmarks

14. Appendix

14.1 File Structure

```
ISCM/
  ISCM.Domain/
    Entities/
      ControlCatalog.cs
      ControlDefinition.cs
      SubControlDefinition.cs
      Evidence.cs
      Finding.cs
      ...
    Enums/
      CheckStatus.cs
      ExpectedValueType.cs
      Operator.cs
      ...
    ValueObjects/
      EvidenceValue.cs
      DurationValue.cs
      SizeValue.cs
      ...
  ISCM.Application/
    Evaluators/
      TypedEvidenceEvaluator.cs
      ControlEvaluator.cs
      Typed/
        IntegerEvaluator.cs
        BooleanEvaluator.cs
        ...
    Interfaces/
      IEvidenceCollector.cs
      IControlEvaluator.cs
      ...
    Services/
      EvidenceAcquisitionService.cs
      RemediationService.cs
      ...
    Validators/
      CatalogValidator.cs
  ISCM.Infrastructure/
    Scanning/
      WindowsHardeningScanner.cs
      WindowsSystemInfoCollector.cs
      Checks/
        BaseHardeningCheck.cs
        PasswordLengthCheck.cs
        AdminAccountCountCheck.cs
        ... (22 total)
  ISCM.Tests/
    Unit/
    Integration/
    ...
```

14.2 Key Metrics

Metric	Value
Lines of Code (estimated)	~15,000
Test Coverage	88 tests, 100% pass rate
Checks	22 migrated (100%)
SubControls	119 defined (100%)
Supported Types	8 (Integer, Boolean, Duration, String, Enum, Collection, RegistryValue, PolicyValue)
Evidence Sources	8 (Registry, PowerShell, WMI, Auditpol, Secedit, NetAccounts, ProcessOutput, Other)

14.3 Dependencies

Category	Packages
Core	.NET 8.0
Testing	xUnit 2.5.7, Moq 4.20.70, FluentAssertions 6.12.0
Platform	Windows-only (System.Management for WMI, Microsoft.Win32.Registry)

14.4 Build Commands

```
# Build all projects
dotnet build

# Run all tests
dotnet test

# Run specific test category
dotnet test --filter "FullyQualifiedName~Integration"

# Run with verbose output
dotnet test --logger "console;verbosity=detailed"
```

14.5 Migration Checklist (for future checks)

- 1. Inherit from BaseHardeningCheck
- 2. Implement CheckId, Name, Category, Severity properties
- 3. Implement CollectEvidenceAsync() method
- 4. Return List with: SubControlId, RawOutput, TypedValue (via EvidenceValue.FromXxx()), Evaluation = CheckStatus.NotScanned
- 5. Add SubControl to ControlCatalog with ExpectedValueType and Operator
- 6. Write integration test
- 7. Verify all tests pass
- 8. Commit with message: feat(phaseXX): migrate CheckName to collector-only pattern (XX.Y)

Document Status: This architecture document reflects the state of ISCM after successful completion of Gate 11. All 22 checks are migrated, the catalog is complete, legacy code has been removed, and 88 tests pass with a 100% success rate.

For questions or contributions, refer to the project repository and the epic branch history described in Section 12.